UNIVERSIDADE DE AVEIRO · ARQUITETURA DE COMPUTADORES I

Testes teóricos

2023 / 2024

Fotografias dos enunciados e chaves publicadas na mesma fonte.

PROVA	ENUNCIADO	CHAVE
Teste 1 · 28/10/2023	2–7	8
Teste 2 · 10/01/2024	9–14	15
Recurso · 26/01/2024	16–22	23

As datas foram confirmadas nas fotografias. A pasta de origem chama-se “Exames2024”, mas o primeiro teste é de outubro de 2023.

Os originais foram conservados na pasta da entrega. Neste PDF, a orientação das fotografias segue a informação do próprio ficheiro. As duas últimas fotografias do recurso mostram a página final com enquadramentos diferentes.

As chaves são imagens da fonte; não foram recalculadas nem verificadas pergunta a pergunta. Algumas fotografias têm cortes, perspetiva ou desfoque que já existiam no original.

Fonte: github.com/luisbfsousa/leci · Consulta: 30/10/2026

Teste 1 · 28/10/2023

Fotografia 1

28/10/2023

Nº Mec.

Notas importantes!

1. Verifique, para todas as questões, qual a resposta correta e assinale com uma cruz a sua escolha na tabela ao lado. Por cada resposta incorreta será descontada, à cotação global, no máximo 1/3 da cotação da respetiva pergunta.
2. Pode usar até um máximo de 4 respostas duplas (por cada dupla: 0 certas desconta até 2/3, 1 certa conta pelo menos 2/3). Se usar mais de 4 duplas, serão aceites as 4 primeiras e as restantes serão consideradas respostas erradas.
3. Durante a realização do teste não é permitida a permanência junto do aluno, mesmo que desligado, de qualquer dispositivo eletrónico não expressamente autorizado (nesta lista incluem-se calculadoras, telemóveis e *smartwatches*). A sua deteção durante a realização do exame implica a imediata anulação do mesmo.

	a	b	c	d
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				
17				
18				
19				
20				
21				

	a	b	c	d
22				
23				
24				
25				
26				
27				
28				
29				
30				
31				
32				

Grupo I

1. Na arquitetura básica de um sistema computacional o *Control Bus* permite:
   a. especificar a natureza da operação efetuada sobre a memória.
   b. identificar, na memória, a origem/destino dos dados transferidos.
   c. transferir dados entre os registos do CPU.
   d. transferir o código máquina das instruções para o *instruction register*.

2. Na arquitetura básica de um sistema computacional, o *Address Bus* permite:
   a. transferir o código máquina das instruções para o *program counter*.
   b. identificar, na memória, a origem/destino da informação transferida.
   c. especificar a natureza das operações efetuadas sobre a memória.
   d. transferir dados entre a memória externa e os registos do CPU.

3. Considere a instrução `bne $6,$2,label` armazenada no endereço de memória `0x00400004`. Se o endereço alvo da instrução for `0x00400020`, o valor dos 16 bits menos significativos do código máquina dessa instrução será:
   a. `0x0018`
   b. `0x0007`
   c. `0x001C`
   d. `0x0006`

4. Suponha que `$3=0xFC47A6E8` e `$2=0x00FFFF00`. A instrução "`xor $6,$3,$2`" produz o seguinte resultado armazenado em `$6`:
   a. `0xFC0000E8`
   b. `0x00B80017`
   c. `0x3047A9E8`
   d. `0xFCB859E8`

5. A arquitetura MIPS <u>não permite</u> realizar operações lógicas e aritméticas sobre o conteúdo de registos da memória externa ou, simultaneamente, sobre o conteúdo de registos internos e de registos da memória externa. Por essa razão:
   a. o número de instruções do seu *set* é superior ao das arquiteturas que permitem esse tipo de operações.
   b. é considerada como sendo do tipo *Load-Store*.
   c. todas as variáveis de um dado programa deverão residir em registos internos.
   d. permite, num mesmo ciclo de relógio, operar sobre a ALU e efetuar uma operação de escrita na memória.

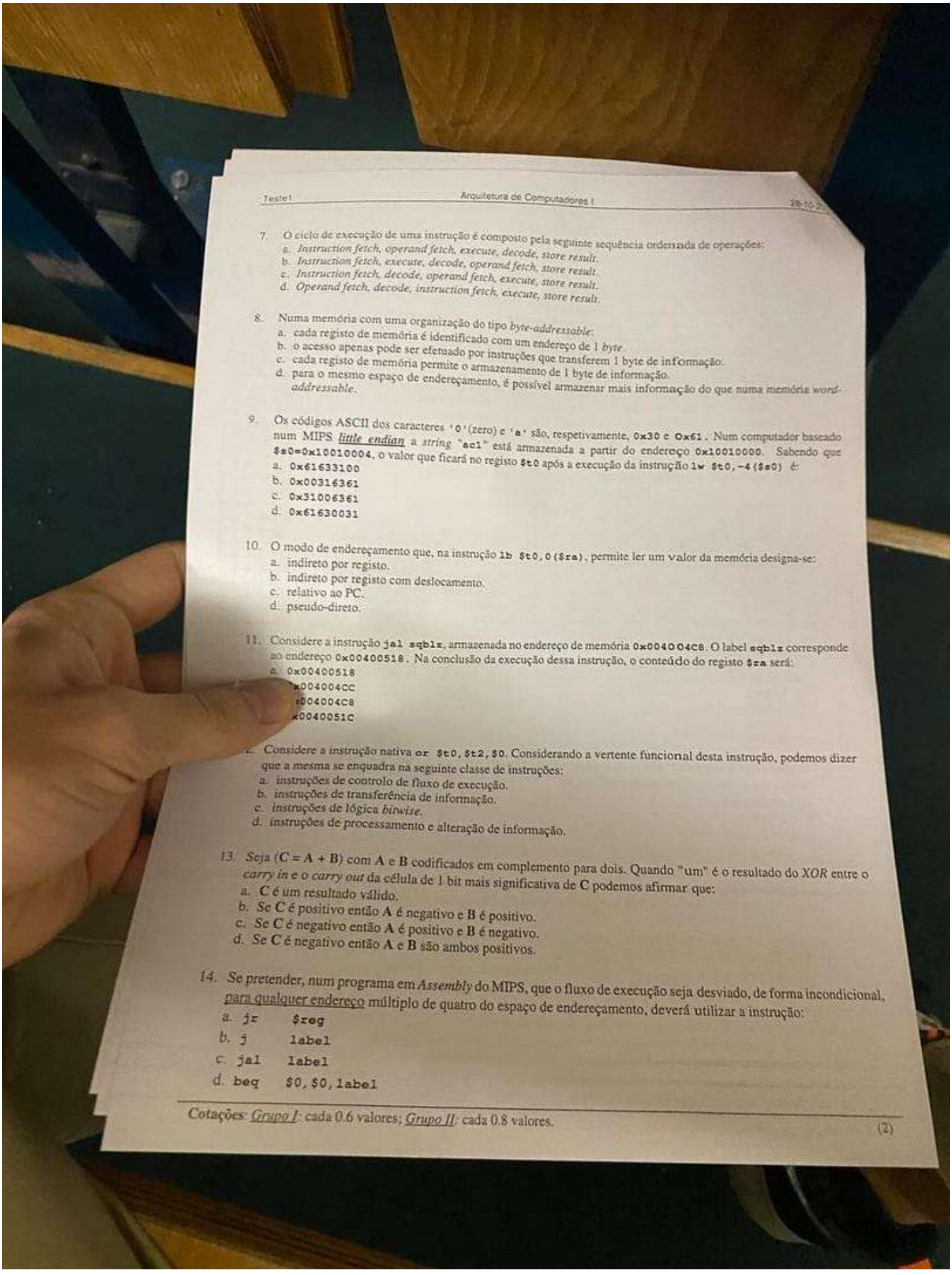
6. Um endereço de memória externa num sistema computacional é:
   a. o valor transferido de um registo do CPU para um registo da memória externa.
   b. o identificador do espaço de endereçamento.
   c. a informação armazenada em cada registo de memória.
   d. um número único que identifica cada registo de memória.

Cotações: <u>*Grupo I*</u>: cada 0.6 valores; <u>*Grupo II*</u>: cada 0.8 valores.

(1)

Teste 1 · 28/10/2023

Fotografia 2



7. O ciclo de execução de uma instrução é composto pela seguinte sequência ordenada de operações:
   a. *Instruction fetch, operand fetch, execute, decode, store result.*
   b. *Instruction fetch, execute, decode, operand fetch, store result.*
   c. *Instruction fetch, decode, operand fetch, execute, store result.*
   d. *Operand fetch, decode, instruction fetch, execute, store result.*

8. Numa memória com uma organização do tipo *byte-addressable*:
   a. cada registo de memória é identificado com um endereço de 1 *byte*.
   b. o acesso apenas pode ser efetuado por instruções que transferem 1 byte de informação.
   c. cada registo de memória permite o armazenamento de 1 byte de informação.
   d. para o mesmo espaço de endereçamento, é possível armazenar mais informação do que numa memória *word-addressable*.

9. Os códigos ASCII dos caracteres `'0'` (zero) e `'a'` são, respetivamente, `0x30` e `0x61`. Num computador baseado num MIPS *little endian* a *string* `"ael"` está armazenada a partir do endereço `0x10010000`. Sabendo que `$s0=0x10010004`, o valor que ficará no registo `$t0` após a execução da instrução `lw $t0,-4($s0)` é:
   a. `0x61633100`
   b. `0x00316361`
   c. `0x31006361`
   d. `0x61630031`

10. O modo de endereçamento que, na instrução `lb $t0,0($ra)`, permite ler um valor da memória designa-se:
    a. indireto por registo.
    b. indireto por registo com deslocamento.
    c. relativo ao PC.
    d. pseudo-direto.

11. Considere a instrução `jal sqblz`, armazenada no endereço de memória `0x004004C8`. O label `sqblz` corresponde ao endereço `0x00400518`. Na conclusão da execução dessa instrução, o conteúdo do registo `$ra` será:
    a. `0x00400518`
    b. `0x004004CC`
    c. `0x004004C8`
    d. `0x0040051C`

12. Considere a instrução nativa `or $t0,$t2,$0`. Considerando a vertente funcional desta instrução, podemos dizer que a mesma se enquadra na seguinte classe de instruções:
    a. instruções de controlo de fluxo de execução.
    b. instruções de transferência de informação.
    c. instruções de lógica *bitwise*.
    d. instruções de processamento e alteração de informação.

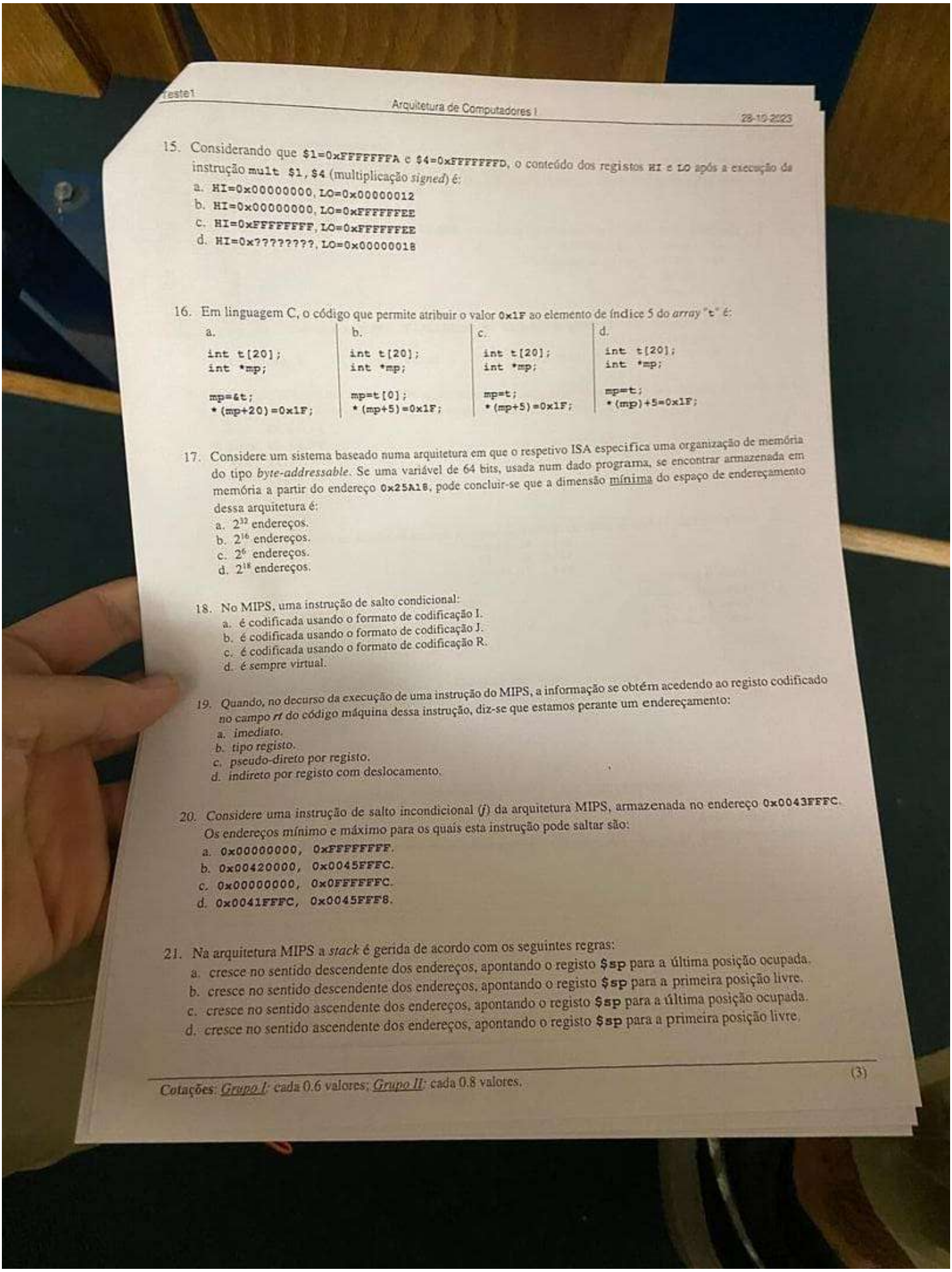
13. Seja (**C** = **A** + **B**) com **A** e **B** codificados em complemento para dois. Quando "um" é o resultado do *XOR* entre o *carry in* e o *carry out* da célula de 1 bit mais significativa de **C** podemos afirmar que:
    a. **C** é um resultado válido.
    b. Se **C** é positivo então **A** é negativo e **B** é positivo.
    c. Se **C** é negativo então **A** é positivo e **B** é negativo.
    d. Se **C** é negativo então **A** e **B** são ambos positivos.

14. Se pretender, num programa em *Assembly* do MIPS, que o fluxo de execução seja desviado, de forma incondicional, para qualquer endereço múltiplo de quatro do espaço de endereçamento, deverá utilizar a instrução:
    a. `jr $reg`
    b. `j label`
    c. `jal label`
    d. `beq $0,$0,label`

Cotações: Grupo I: cada 0.6 valores; Grupo II: cada 0.8 valores.

Teste 1 · 28/10/2023

Fotografia 3



15. Considerando que `$1=0xFFFFFFFA` e `$4=0xFFFFFFFD`, o conteúdo dos registos `HI` e `LO` após a execução da instrução `mult $1,$4` (multiplicação *signed*) é:
    a. `HI=0x00000000, LO=0x00000012`
    b. `HI=0x00000000, LO=0xFFFFFFEE`
    c. `HI=0xFFFFFFFF, LO=0xFFFFFFEE`
    d. `HI=0x????????, LO=0x00000018`

16. Em linguagem C, o código que permite atribuir o valor `0x1F` ao elemento de índice 5 do *array* "`t`" é:

a.
```
int t[20];
int *mp;

mp=&t;
*(mp+20)=0x1F;
```

b.
```
int t[20];
int *mp;

mp=t[0];
*(mp+5)=0x1F;
```

c.
```
int t[20];
int *mp;

mp=t;
*(mp+5)=0x1F;
```

d.
```
int t[20];
int *mp;

mp=t;
*(mp)+5=0x1F;
```

17. Considere um sistema baseado numa arquitetura em que o respetivo ISA especifica uma organização de memória do tipo *byte-addressable*. Se uma variável de 64 bits, usada num dado programa, se encontrar armazenada em memória a partir do endereço `0x25A18`, pode concluir-se que a dimensão mínima do espaço de endereçamento dessa arquitetura é:
    a. 2^{32} endereços.
    b. 2^{16} endereços.
    c. 2^{6} endereços.
    d. 2^{18} endereços.

18. No MIPS, uma instrução de salto condicional:
    a. é codificada usando o formato de codificação I.
    b. é codificada usando o formato de codificação J.
    c. é codificada usando o formato de codificação R.
    d. é sempre virtual.

19. Quando, no decurso da execução de uma instrução do MIPS, a informação se obtém acedendo ao registo codificado no campo *rt* do código máquina dessa instrução, diz-se que estamos perante um endereçamento:
    a. imediato.
    b. tipo registo.
    c. pseudo-direto por registo.
    d. indireto por registo com deslocamento.

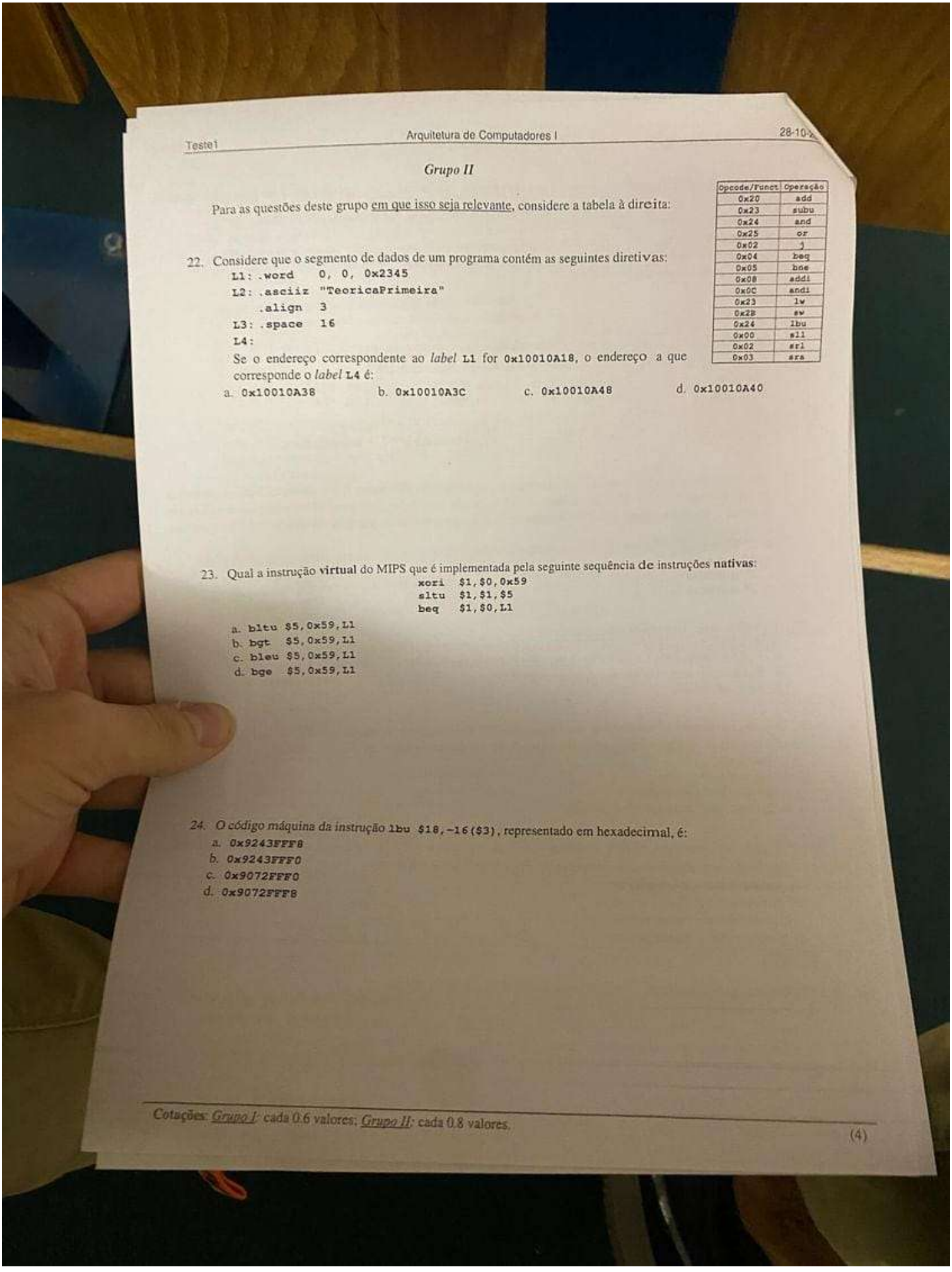
20. Considere uma instrução de salto incondicional (*j*) da arquitetura MIPS, armazenada no endereço `0x0043FFFC`. Os endereços mínimo e máximo para os quais esta instrução pode saltar são:
    a. `0x00000000, 0xFFFFFFFF`.
    b. `0x00420000, 0x0045FFFC`.
    c. `0x00000000, 0x0FFFFFFC`.
    d. `0x0041FFFC, 0x0045FFF8`.

21. Na arquitetura MIPS a *stack* é gerida de acordo com os seguintes regras:
    a. cresce no sentido descendente dos endereços, apontando o registo `$sp` para a última posição ocupada.
    b. cresce no sentido descendente dos endereços, apontando o registo `$sp` para a primeira posição livre.
    c. cresce no sentido ascendente dos endereços, apontando o registo `$sp` para a última posição ocupada.
    d. cresce no sentido ascendente dos endereços, apontando o registo `$sp` para a primeira posição livre.

Cotações: *Grupo I*: cada 0.6 valores; *Grupo II*: cada 0.8 valores. (3)

Teste 1 · 28/10/2023

Fotografia 4



Teste1 Arquitetura de Computadores I 28-10-2

Grupo II

Para as questões deste grupo em que isso seja relevante, considere a tabela à direita:

Opcode/Funct	Operação
0x20	add
0x23	subu
0x24	and
0x25	or
0x02	j
0x04	beq
0x05	bne
0x08	addi
0x0C	andi
0x23	lw
0x28	sw
0x24	lbu
0x00	sll
0x02	srl
0x03	sra

22. Considere que o segmento de dados de um programa contém as seguintes diretivas:

```
L1: .word   0, 0, 0x2345
L2: .asciiz "TeoricaPrimeira"
    .align  3
L3: .space  16
L4:
```

Se o endereço correspondente ao *label* `L1` for `0x10010A18`, o endereço a que corresponde o *label* `L4` é:

a. `0x10010A38` b. `0x10010A3C` c. `0x10010A48` d. `0x10010A40`

23. Qual a instrução **virtual** do MIPS que é implementada pela seguinte sequência de instruções **nativas**:

```
xori  $1,$0,0x59
sltu  $1,$1,$5
beq   $1,$0,L1
```

a. `bltu $5,0x59,L1`
b. `bgt  $5,0x59,L1`
c. `bleu $5,0x59,L1`
d. `bge  $5,0x59,L1`

24. *O código máquina da instrução* `lbu $18,-16($3)`, representado em hexadecimal, é:

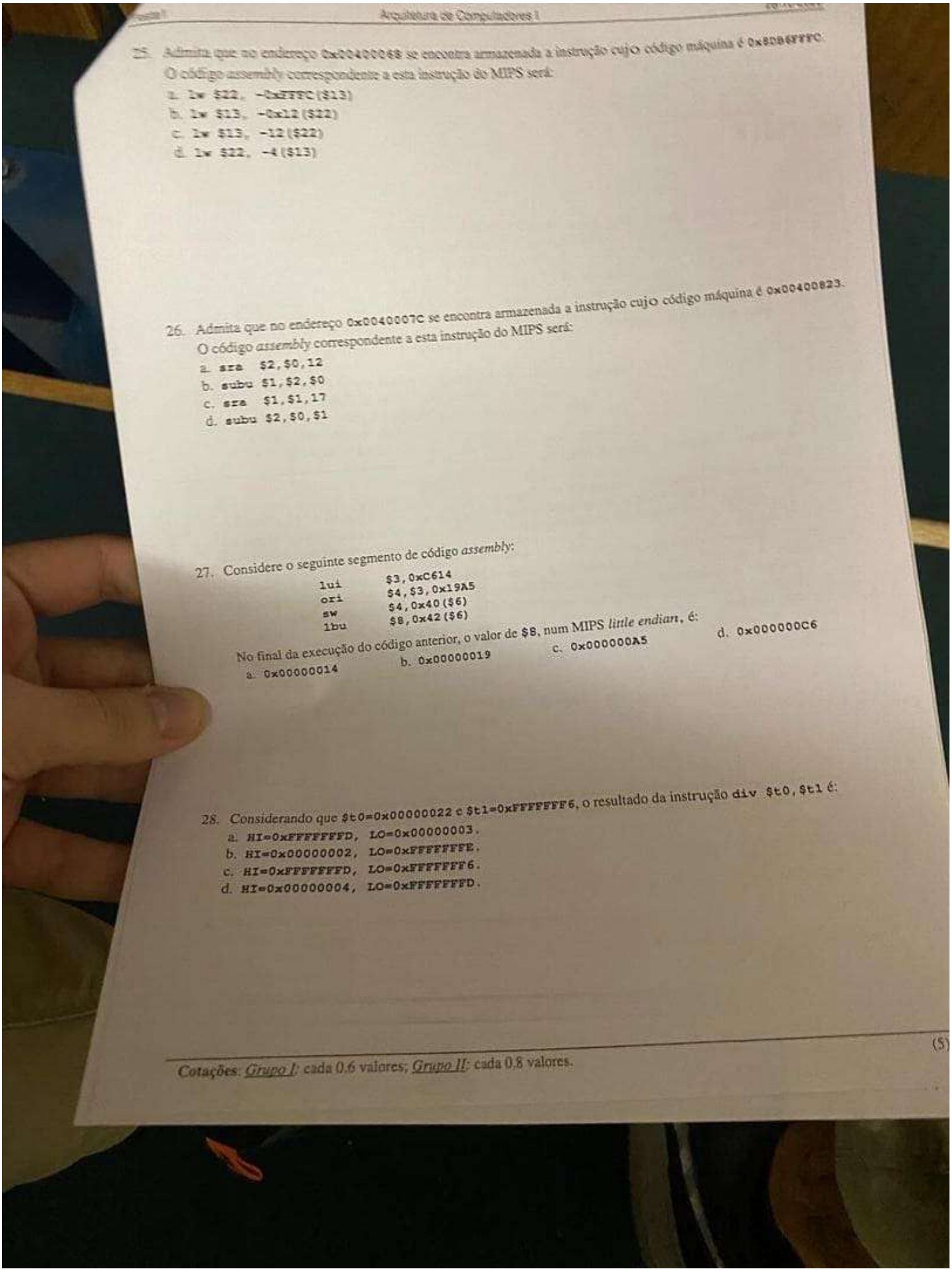
a. `0x9243FFF8`
b. `0x9243FFF0`
c. `0x9072FFF0`
d. `0x9072FFF8`

Cotações: *Grupo I*: cada 0.6 valores; *Grupo II*: cada 0.8 valores.

(4)

Teste 1 · 28/10/2023

Fotografia 5



Arquitetura de Computadores I

25. Admita que no endereço `0x00400068` se encontra armazenada a instrução cujo código máquina é `0x8D86FFFC`. O código *assembly* correspondente a esta instrução do MIPS será:
   a. `lw $22, -0xFFFC($13)`
   b. `lw $13, -0x12($22)`
   c. `lw $13, -12($22)`
   d. `lw $22, -4($13)`

26. Admita que no endereço `0x0040007C` se encontra armazenada a instrução cujo código máquina é `0x00400823`. O código *assembly* correspondente a esta instrução do MIPS será:
   a. `sra $2,$0,12`
   b. `subu $1,$2,$0`
   c. `sra $1,$1,17`
   d. `subu $2,$0,$1`

27. Considere o seguinte segmento de código *assembly*:

```
lui   $3,0xC614
ori   $4,$3,0x19A5
sw    $4,0x40($6)
lbu   $8,0x42($6)
```

No final da execução do código anterior, o valor de `$8`, num MIPS *little endian*, é:
   a. `0x00000014`
   b. `0x00000019`
   c. `0x000000A5`
   d. `0x000000C6`

28. Considerando que `$t0=0x00000022` e `$t1=0xFFFFFFF6`, o resultado da instrução `div $t0,$t1` é:
   a. `HI=0xFFFFFFFD, LO=0x00000003.`
   b. `HI=0x00000002, LO=0xFFFFFFFE.`
   c. `HI=0xFFFFFFFD, LO=0xFFFFFFF6.`
   d. `HI=0x00000004, LO=0xFFFFFFFD.`

Cotações: *Grupo I*: cada 0.6 valores; *Grupo II*: cada 0.8 valores. (5)

29. O resultado da instrução `mult $t0,$t1` é representável em 32 bits se:
    a. `HI` for diferente de zero.
    b. `HI=0xFFFFFFFF`.
    c. `HI` for uma extensão com sinal de `LO`.
    d. `HI=0x00000000`.

30. Os processadores da arquitetura hipotética **AC1TEO** implementam um total de **59** instruções. Todas as instruções são codificadas em 32 bits, num formato com 5 campos: *opcode*, três campos para identificar registos internos e um campo para codificar valores imediatos na gama `[-4096, +4095]`. Pode concluir-se, portanto:
    a. que o número de registos internos é **8** e o campo *opcode* tem **8** bits.
    b. que o número de registos internos é **32** e o campo *opcode* tem **6** bits.
    c. que o número de registos internos é **16** e o campo *opcode* tem **6** bits.
    d. que o número de registos internos é **16** e o campo *opcode* tem **7** bits.

31. Ao executar o trecho de código da figura ao lado:
    a. será apresentada no ecrã a *string* "567".
    b. será gerada de uma exceção na instrução "`lw`" devido a uma tentativa de acesso não alinhado à memória.
    c. será apresentada no ecrã a *string* "3456".
    d. não é possível saber o resultado da instrução `syscall` uma vez que o espaço de memória que começa no *label* "`buf`" não é inicializado.

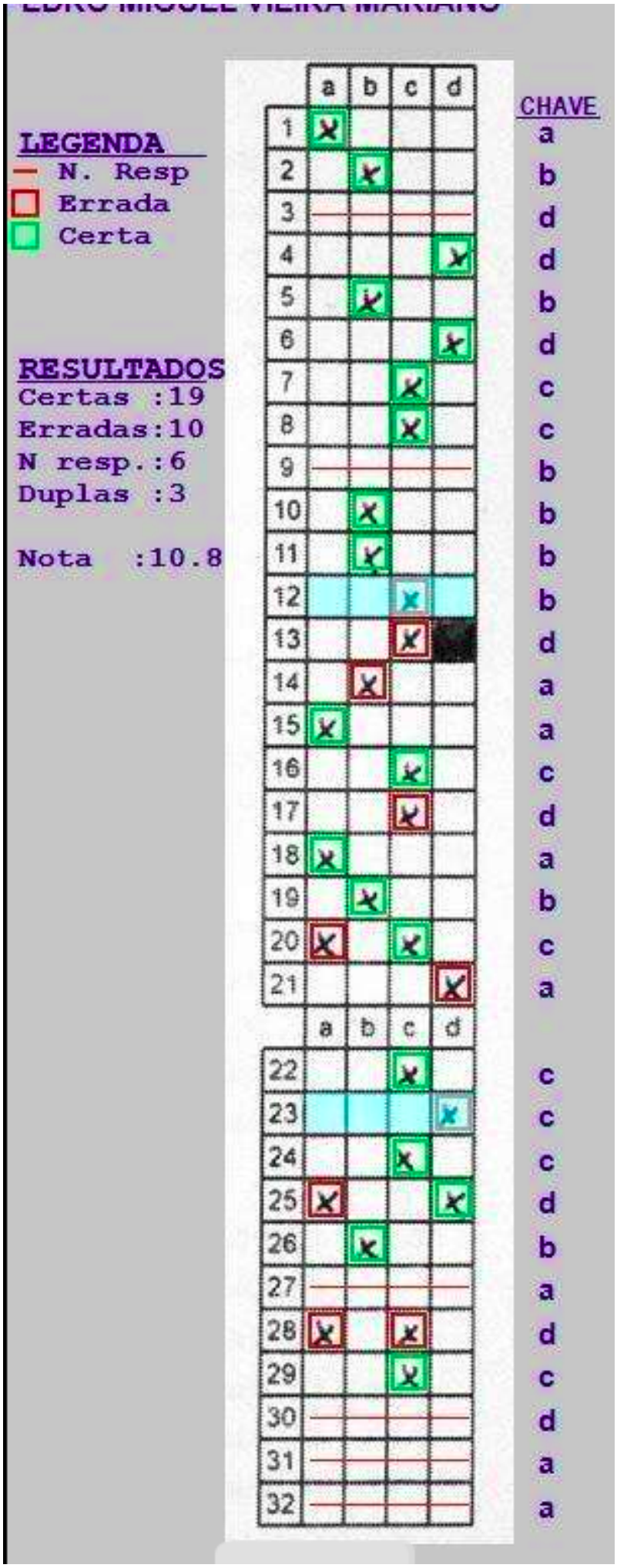
```
        .data
        .align 3
str:    .byte '1','2','3','4','5','6','7',0
buf:    .space 4
        .text
        .globl main
main:   la    $t0, str
        addiu $t0, $t0, 3
        lw    $t1, 1($t0)
        sw    $t1, 5($t0)
        li    $v0, 4
        la    $a0, buf
        syscall              # print_string(buf);
        jr    $ra
```

32. Considere que no endereço de memória `0x5C271934` se encontra a instrução cujo código máquina é: `0x0841ADF3`. Estamos neste caso a referir-nos à instrução:
    a. `j 0x5006B7CC`
    b. `j 0x0806B7CC`
    c. `j 0x0841ADFC`
    d. `j 0x08455820`

Cotações: Grupo I: cada 0.6 valores; Grupo II: cada 0.8 valores. (6)

Teste 1 · 28/10/2023

Chave de respostas publicada com o teste



Notas importantes!

1. Verifique, para todas as questões, qual a resposta correta e assinale com uma cruz a sua escolha na tabela ao lado. Por cada resposta incorreta será descontada, à cotação global, no máximo 1/3 da cotação da respetiva pergunta.
2. Pode usar até um máximo de 4 respostas duplas (por cada dupla: 0 certas desconta até 2/3, 1 certa conta 2/3). Se usar mais de 4 duplas, serão aceites as 4 primeiras e as restantes serão consideradas respostas erradas.
3. Durante a realização do teste não é permitida a permanência junto do aluno, mesmo que desligado, de qualquer dispositivo eletrónico não expressamente autorizado (nesta lista incluem-se calculadoras, telemóveis e *smartwatches/smartbands*). A sua deteção durante a realização do exame implica a imediata anulação do mesmo.

N Mec

	a	b	c	d
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				
17				
18				
19				
20				
21				
	a	b	c	d
22				
23				
24				
25				
26				
27				
28				
29				
30				
31				
32				

Grupo I

1. No decurso da execução de uma instrução sobre uma implementação *single cycle* da arquitetura MIPS, o valor presente na saída do registo PC corresponde:
   a. ao endereço de retorno das funções.
   b. ao endereço da instrução em curso.
   c. ao endereço da instrução seguinte.
   d. ao endereço alvo se a instrução for um `j`.

2. Considere um sistema computacional em que existe uma memória do tipo ROM onde se encontram armazenados programas e uma memória do tipo RAM onde se encontram armazenados dados. Residindo as duas memórias no mesmo espaço de endereçamento, pode afirmar-se que estamos perante uma arquitetura:
   a. do tipo *Harvard*.
   b. mista *Harvard/von Neumann*.
   c. do tipo *von Neumann*.
   d. do tipo *Harvard* com armazenamento não volátil de programas.

3. Um endereço de memória externa num sistema computacional é:
   a. um número único que identifica cada posição de memória.
   b. o tipo de operação realizada sobre a memória.
   c. a informação armazenada em cada posição de memória.
   d. o valor transferido de um registo do CPU para uma posição de memória.

4. Considere uma arquitetura *von Neumann*, com um barramento de endereços de 24 *bits* e uma organização de memória *byte-addressable*. A dimensão máxima, expressa em **bytes**, que um programa a executar neste sistema (instruções + dados + *stack*) poderá ter, será:
   a. 16 MByte
   b. 256 MByte
   c. 64 MByte
   d. 1 GByte

5. No formato de vírgula flutuante IEEE 754, precisão dupla, um expoente positivo é codificado:
   a. com um valor superior a 0 e inferior a 128.
   b. com um valor superior a 0 e inferior a 1024.
   c. com um valor superior a 1023 e inferior a 2047.
   d. com um valor que tem o bit mais significativo a zero.

6. Numa implementação *multy-cycle* de um processador MIPS, o sinal *MemRead*, quando ativo, habilita a leitura da memória:
   a. de forma assíncrona e imediata.
   b. de forma assíncrona no próximo ciclo do sinal de relógio.
   c. no próximo flanco ativo do sinal de relógio.
   d. durante o tempo correspondente a 1 ciclo de relógio.

Cotações: Grupo I: cada 0.6 valores; Grupo II: cada 0.8 valor

7. Uma arquitetura hipotética baseada num *datapath single cycle* em que os operandos das instruções aritméticas e lógicas podem residir em registos internos ou na memória externa, pode ser classificada como:
   a. uma arquitetura Harvard do tipo "*Load-store*".
   b. uma arquitetura Von Neumann do tipo "*Register-Memory*".
   c. uma arquitetura Von Neumann do tipo "*Load-store*".
   d. uma arquitetura Harvard do tipo "*Register-Memory*".

8. Considere que o segmento de dados de um programa contém as seguintes diretivas:

```
L1: .word   0, 0
L2: .asciiz "TesteAC1: "
    .align  3
L3: .space  12
L4:
```

   Se o endereço correspondente ao *label* **L1** for **0x10010A0C**, o endereço do *label* **L4** é:
   a. **0x10010A24** b. **0x10010A40** c. **0x10010A3C** d. **0x10010A2C**

9. Suponha que um *datapath single cycle* está a executar a instrução **add $3, $4, $5**. As operações que serão realizadas na próxima transição ativa do sinal de relógio são:
   a. conclusão da operação na ALU e escrita no banco de registos.
   b. escrita no banco de registos e atualização do estado da unidade de controlo.
   c. escrita no banco de registos e escrita no PC.
   d. escrita no banco de registos no endereço que consta do campo **rt**.

10. O valor à saída da ALU, na conclusão do segundo ciclo de relógio num *datapath multi-cycle*, durante a execução da instrução com o código máquina **0x10430024**, supondo que o valor à saída do registo *Program Counter* é **0x00400034**, é:
    a. **0x00400058**
    b. **0x004000C8**
    c. **0x004000C4**
    d. **0x0040005C**

11. Numa implementação *single cycle* da arquitetura MIPS, a frequência máxima de operação é de 2GHz (para os atrasos de propagação a seguir indicados). O atraso máximo que pode ocorrer nas operações da ALU será:

    Memórias externas: leitura – 175ps, escrita – 150ps; *File register*: leitura – 25ps, Escrita – 35ps; *Unidade de Controlo*: 10ps; *Somadores*: 50ps; Outros: 0ns; *Escrita noutros elementos de estado*: 20ps.

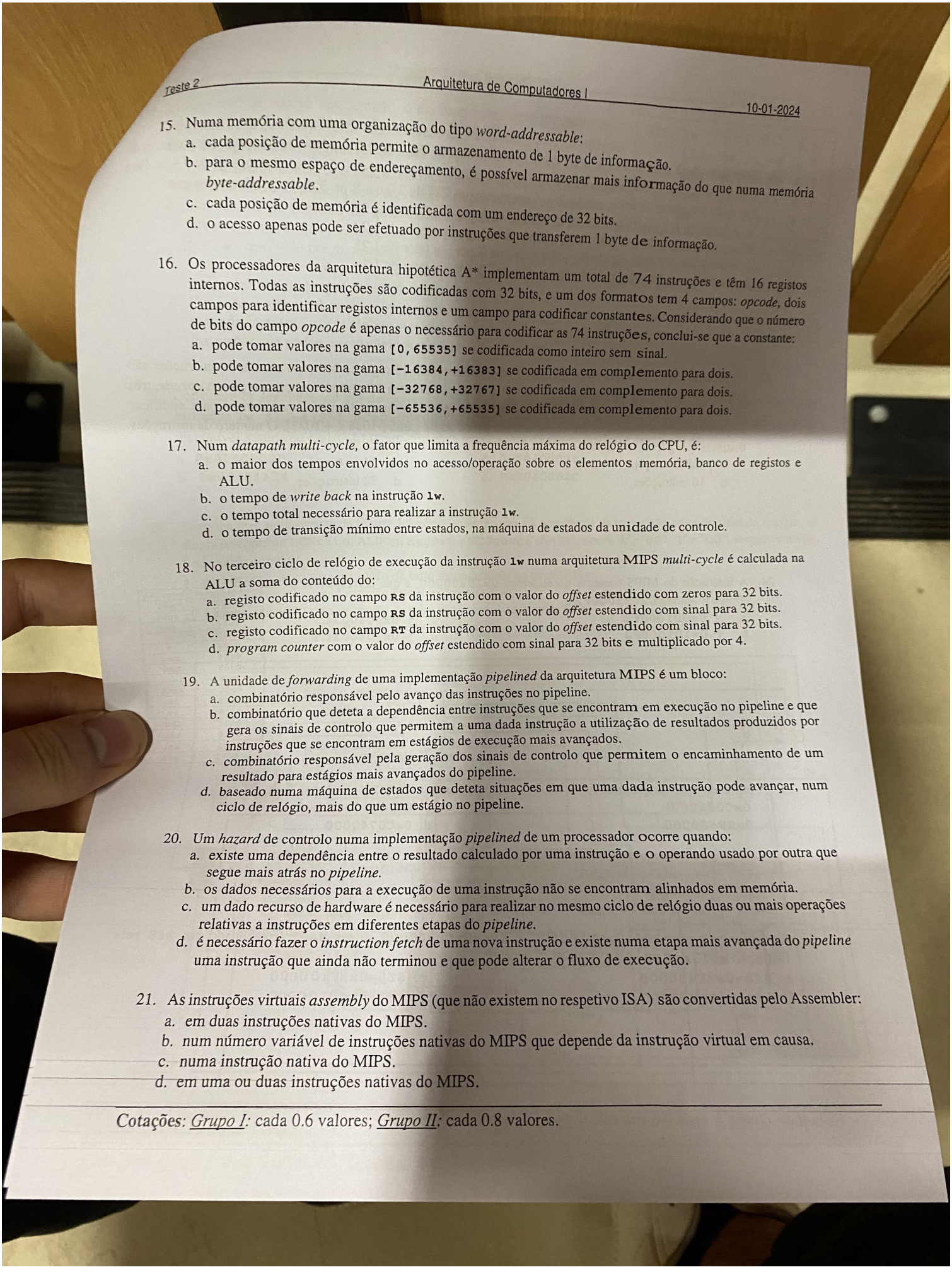
    a. teria de ser inferior a 0ps
    b. 80ps
    c. 90ps
    d. 110ps

12. Na implementação *multi-cycle* de um processador MIPS, na execução da instrução **xor $1,$2,$3**, a unidade de controlo ativa o sinal:
    a. **MemWrite** no primeiro ciclo de relógio.
    b. **PCWrite** no segundo ciclo de relógio.
    c. **RegWrite** no quarto ciclo de relógio.
    d. **IRWrite** no segundo ciclo de relógio.

13. Se pretender que, num programa do MIPS, o fluxo de execução seja desviado de forma incondicional para qualquer endereço múltiplo de quatro do espaço de endereçamento, deverá utilizar a instrução do tipo:
    a. **jr $reg**
    b. **j label**
    c. **jal label**
    d. **beq $0,$0,label**

14. Suponha a mantissa com o valor binário **1.00111000**. Admitindo que a parte fracionária da mantissa deve ser armazenada em 4 bits, o valor final da mantissa após arredondamento para o par mais próximo será:
    a. **1.0010**
    b. **1.0100**
    c. **1.0011**
    d. **1.0101**

Cotações: *Grupo I:* cada 0.6 valores; *Grupo II:* cada 0.8 valores.



15. Numa memória com uma organização do tipo *word-addressable*:
    a. cada posição de memória permite o armazenamento de 1 byte de informação.
    b. para o mesmo espaço de endereçamento, é possível armazenar mais informação do que numa memória *byte-addressable*.
    c. cada posição de memória é identificada com um endereço de 32 bits.
    d. o acesso apenas pode ser efetuado por instruções que transferem 1 byte de informação.

16. Os processadores da arquitetura hipotética A* implementam um total de 74 instruções e têm 16 registos internos. Todas as instruções são codificadas com 32 bits, e um dos formatos tem 4 campos: *opcode*, dois campos para identificar registos internos e um campo para codificar constantes. Considerando que o número de bits do campo *opcode* é apenas o necessário para codificar as 74 instruções, conclui-se que a constante:
    a. pode tomar valores na gama `[0,65535]` se codificada como inteiro sem sinal.
    b. pode tomar valores na gama `[-16384,+16383]` se codificada em complemento para dois.
    c. pode tomar valores na gama `[-32768,+32767]` se codificada em complemento para dois.
    d. pode tomar valores na gama `[-65536,+65535]` se codificada em complemento para dois.

17. Num *datapath multi-cycle*, o fator que limita a frequência máxima do relógio do CPU, é:
    a. o maior dos tempos envolvidos no acesso/operação sobre os elementos memória, banco de registos e ALU.
    b. o tempo de *write back* na instrução `lw`.
    c. o tempo total necessário para realizar a instrução `lw`.
    d. o tempo de transição mínimo entre estados, na máquina de estados da unidade de controle.

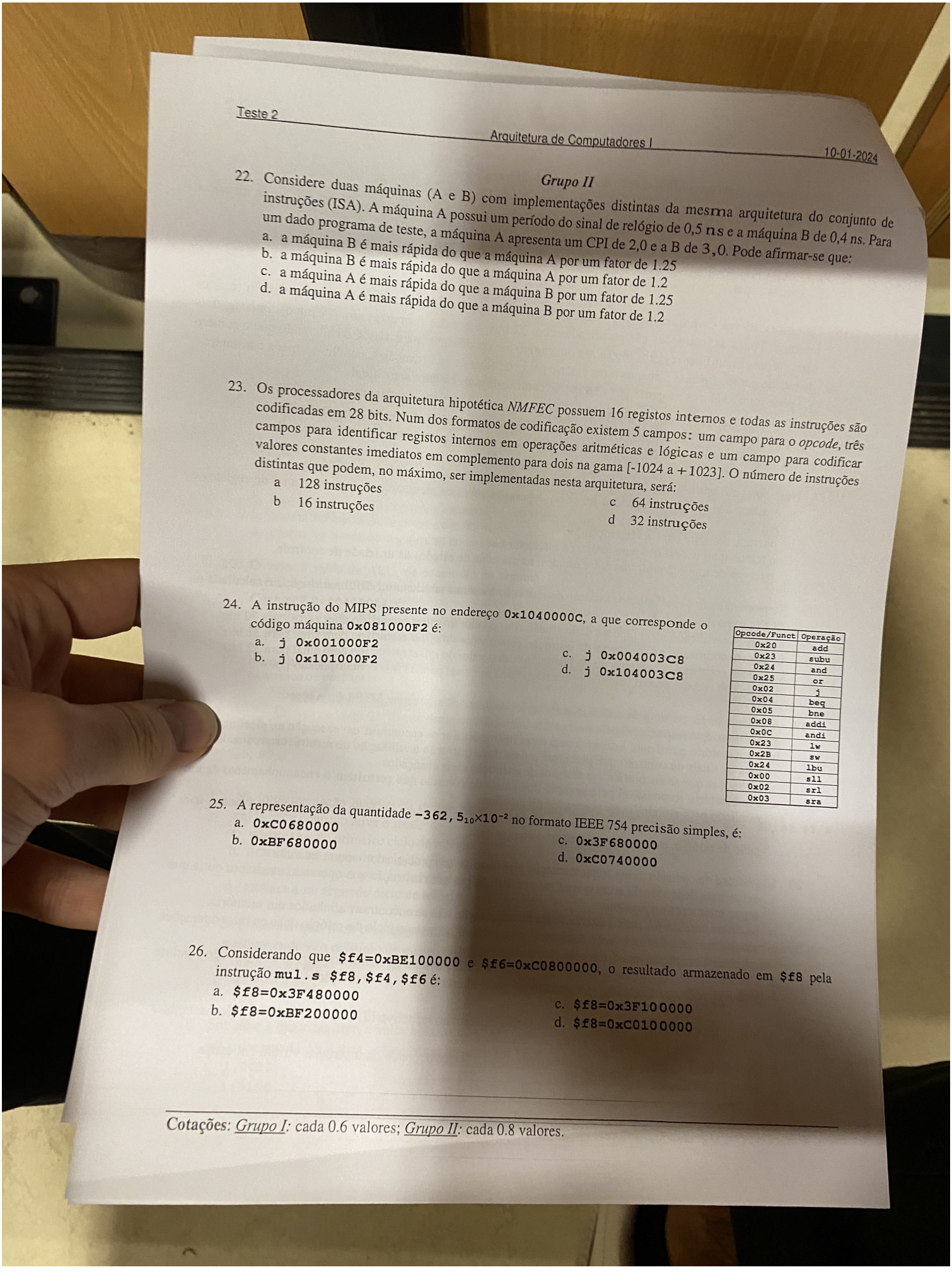
18. No terceiro ciclo de relógio de execução da instrução `lw` numa arquitetura MIPS *multi-cycle* é calculada na ALU a soma do conteúdo do:
    a. registo codificado no campo `RS` da instrução com o valor do *offset* estendido com zeros para 32 bits.
    b. registo codificado no campo `RS` da instrução com o valor do *offset* estendido com sinal para 32 bits.
    c. registo codificado no campo `RT` da instrução com o valor do *offset* estendido com sinal para 32 bits.
    d. *program counter* com o valor do *offset* estendido com sinal para 32 bits e multiplicado por 4.

19. A unidade de *forwarding* de uma implementação *pipelined* da arquitetura MIPS é um bloco:
    a. combinatório responsável pelo avanço das instruções no pipeline.
    b. combinatório que deteta a dependência entre instruções que se encontram em execução no pipeline e que gera os sinais de controlo que permitem a uma dada instrução a utilização de resultados produzidos por instruções que se encontram em estágios de execução mais avançados.
    c. combinatório responsável pela geração dos sinais de controlo que permitem o encaminhamento de um resultado para estágios mais avançados do pipeline.
    d. baseado numa máquina de estados que deteta situações em que uma dada instrução pode avançar, num ciclo de relógio, mais do que um estágio no pipeline.

20. Um *hazard* de controlo numa implementação *pipelined* de um processador ocorre quando:
    a. existe uma dependência entre o resultado calculado por uma instrução e o operando usado por outra que segue mais atrás no *pipeline*.
    b. os dados necessários para a execução de uma instrução não se encontram alinhados em memória.
    c. um dado recurso de hardware é necessário para realizar no mesmo ciclo de relógio duas ou mais operações relativas a instruções em diferentes etapas do *pipeline*.
    d. é necessário fazer o *instruction fetch* de uma nova instrução e existe numa etapa mais avançada do *pipeline* uma instrução que ainda não terminou e que pode alterar o fluxo de execução.

21. As instruções virtuais *assembly* do MIPS (que não existem no respetivo ISA) são convertidas pelo Assembler:
    a. em duas instruções nativas do MIPS.
    b. num número variável de instruções nativas do MIPS que depende da instrução virtual em causa.
    c. numa instrução nativa do MIPS.
    d. em uma ou duas instruções nativas do MIPS.

Cotações: Grupo I: cada 0.6 valores; *Grupo II:* cada 0.8 valores.



Grupo II

22. Considere duas máquinas (A e B) com implementações distintas da mesma arquitetura do conjunto de instruções (ISA). A máquina A possui um período do sinal de relógio de 0,5 ns e a máquina B de 0,4 ns. Para um dado programa de teste, a máquina A apresenta um CPI de 2,0 e a B de 3,0. Pode afirmar-se que:
    a. a máquina B é mais rápida do que a máquina A por um fator de 1.25
    b. a máquina B é mais rápida do que a máquina A por um fator de 1.2
    c. a máquina A é mais rápida do que a máquina B por um fator de 1.25
    d. a máquina A é mais rápida do que a máquina B por um fator de 1.2

23. Os processadores da arquitetura hipotética *NMFEC* possuem 16 registos internos e todas as instruções são codificadas em 28 bits. Num dos formatos de codificação existem 5 campos: um campo para o *opcode*, três campos para identificar registos internos em operações aritméticas e lógicas e um campo para codificar valores constantes imediatos em complemento para dois na gama [-1024 a +1023]. O número de instruções distintas que podem, no máximo, ser implementadas nesta arquitetura, será:
    a 128 instruções
    b 16 instruções
    c 64 instruções
    d 32 instruções

24. A instrução do MIPS presente no endereço `0x1040000C`, a que corresponde o código máquina `0x081000F2` é:
    a. `j 0x001000F2`
    b. `j 0x101000F2`
    c. `j 0x004003C8`
    d. `j 0x104003C8`

Opcode/Funct	Operação
0x20	add
0x23	subu
0x24	and
0x25	or
0x02	j
0x04	beq
0x05	bne
0x08	addi
0x0C	andi
0x23	lw
0x2B	sw
0x24	lbu
0x00	sll
0x02	srl
0x03	sra

25. A representação da quantidade $-362,5_{10}\times10^{-2}$ no formato IEEE 754 precisão simples, é:
    a. `0xC0680000`
    b. `0xBF680000`
    c. `0x3F680000`
    d. `0xC0740000`

26. Considerando que `$f4=0xBE100000` e `$f6=0xC0800000`, o resultado armazenado em `$f8` pela instrução `mul.s $f8,$f4,$f6` é:
    a. `$f8=0x3F480000`
    b. `$f8=0xBF200000`
    c. `$f8=0x3F100000`
    d. `$f8=0xC0100000`

Cotações: Grupo I: cada 0.6 valores; *Grupo II:* cada 0.8 valores.

27. O segmento de código apresentado, a executar sobre uma implementação *pipelined* da arquitetura MIPS com unidade de *forwarding* para os estágios **EX** e **ID**, apresenta os seguintes *hazards*:

```
L1: lw   $3,0($4)  # 1
    addi $4,$4,-4  # 2
    add  $5,$5,$3  # 3
    beq  $4,$0,L1  # 4
    sw   $3,4($4)  # 5
```

a. Um *hazard* de controlo na quarta instrução e *hazards* de dados na terceira e quarta instruções que podem ser resolvidos por *forwarding* para **EX**.
b. Um *hazard* de controlo na quarta instrução, um *hazard* de dados na terceira instrução que pode ser resolvido por *forwarding* para **EX** e um *hazard* de dados na quarta instrução que pode ser resolvido por *forwarding* para **ID**.
c. Um *hazard* de controlo na quarta instrução e *hazards* de dados nas segunda, terceira e quarta instruções que podem ser resolvidos por *forwarding* para **EX**.
d. Um *hazard* estrutural na primeira instrução, um *hazard* de controlo na quinta instrução, um *hazard* de dados na segunda instrução que pode ser resolvido por *forwarding* para **ID** e *hazards* de dados nas terceira e quarta instruções que podem ser resolvidos por *forwarding* para **EX**.

28. Considere os seguintes valores de elementos de estado do *datapath* da Figura 1, na segunda fase de execução da instrução `lw $7,-4($5)`: `PC=0x00400030`, `$5=0x00001A28`, `$7=0x0O003C24`. Nesse cenário, o valor disponível à saída do registo **ALUOut** durante a terceira fase de execução dessa instrução é:

a. `0x00400020`
b. `0x00001A24`
c. `0x00400024`
d. `0x0040002C`

29. Suponha que, no *datapath* da Figura 1, imediatamente antes da ocorrência da transição ativa do sinal de relógio, os sinais de controlo **PCWrite**, **ALUOp**, **RegDst** e **PCSource** têm os valores '1', "00", '1' e "00", respetivamente. Pode então concluir-se que:
a. está em execução uma instrução de salto incondicional na terceira fase.
b. está em execução uma instrução tipo R na terceira fase.
c. está em execução uma instrução de salto condicional na terceira fase.
d. está em execução uma instrução na primeira fase.

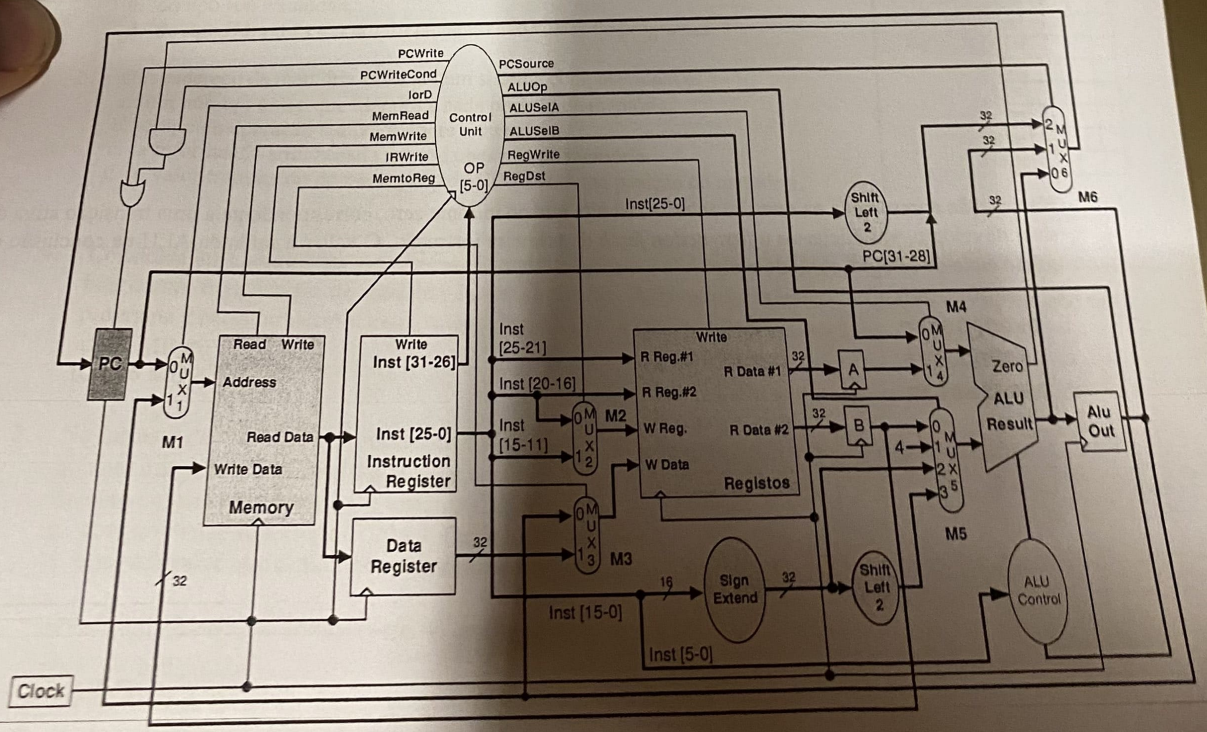


Figura 1.

Cotações: *Grupo I:* cada 0.6 valores; *Grupo II:* cada 0.8 valores.

Cotações: *Grupo I:* cada 0.6 valores; *Grupo II:* cada 0.8 valor

30. Considere uma implementação *pipelined* da arquitetura MIPS. O *hazard* de dados existente na sequência de instruções "`lw $1,0($2)`" seguida de "`beq $1,$3,target`" pode ser resolvido:
    a. com *stall* durante 2 ciclos de relógio.
    b. sem recurso a *stalling*, com *forwarding* de **EX/MEM** para **EX**.
    c. com *stall* durante 1 ciclo de relógio seguido de *forwarding* de **MEM/WB** para **EX**.
    d. com *stall* durante 1 ciclo de relógio seguido de *forwarding* de **EX/MEM** para **ID**.

31. Tome como referência as tabelas a seguir apresentadas. Admita que o valor presente no registo **PC** é **0x0040005C** e corresponde ao endereço da primeira instrução do programa. Considere ainda a implementação *pipelined* da arquitetura MIPS que estudou nas aulas, com *delayed-branch* e *forwarding* para **EX** (**MEM/WB→EX**, **EX/MEM→EX**) e para **ID** (**EX/MEM→ID**).

Endereço	Dados
...	...
0x684C	0x093B863D
0x6848	0x14A0C373
0x6844	0xC31748FE
0x6840	0x601F3212
0x683C	0x26B51E8C
0x6838	0x0F193ABA
0x6834	0x0B506C98
0x6830	0x03C12972

```
L0: addi $4,$0,0x0005
    xor  $2,$2,$2
    addi $3,$0,0x684C
L1: lw   $5,0($3)
    and  $6,$5,$4
    beq  $6,$0,L2
    add  $2,$2,$5
    j    L1
    addi $3,$3,-8
L2: sw   $2,-64($3)
```

A execução completa do trecho de código fornecido, desde o *instruction fetch* da instrução referenciada pelo *label* **L0** até à conclusão da instrução referenciada pelo *label* **L2**, demora:
    a. 14 ciclos de relógio.
    b. 38 ciclos de relógio.
    c. 54 ciclos de relógio.
    d. 42 ciclos de relógio.

32. Na situação apresentada na questão anterior, admita que no instante zero, correspondente a uma transição ativa do sinal de relógio, vai iniciar-se o *instruction fetch* da primeira instrução. O valor à saída da ALU na conclusão do quinto ciclo de relógio, contado a partir do instante zero, é:
    a. **0x00000000**
    b. **0x0000684C**
    c. **0x00000005**
    d. **0x093B863D**

Cotações: Grupo I: cada 0.6 valores

Teste 2 · 10/01/2024

Chave de respostas publicada com o teste

Universidade de Aveiro
Dep. de Electrónica, Telecomunicações e Informática
Arquitetura de Computadores I

ACT - Recurso 26/01/2024

Nome: _____ N.Mec _____

Notas importantes!

1. Verifique, para todas as questões, qual a resposta correta e assinale com uma cruz a sua escolha na tabela ao lado. Por cada resposta incorreta será descontada, à cotação global, no máximo 1/3 da cotação da respetiva pergunta.
2. Pode usar até um máximo de 4 respostas duplas (por cada dupla: 0 certas desconta até 2/3, 1 certa conta até 7/8). Se usar mais de 4 duplas, serão aceites as 4 primeiras e as restantes serão consideradas respostas erradas.
3. Durante a realização do teste não é permitida a permanência junto do aluno, **mesmo que desligado**, de qualquer dispositivo eletrónico não expressamente autorizado (nesta lista incluem-se calculadoras, telemóveis, *smartwatches e smartbands*). A sua deteção durante a realização do exame **implica a imediata anulação do mesmo.**

	a	b	c	d
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				
17				
18				
19				
20				
21				
	a	b	c	d
22				
23				
24				
25				
26				
27				
28				
29				
30				
31				
32				

Grupo I

1. O principal modo de endereçamento utilizado na instrução `beq $t0,$t1,label` é:
   a. relativo ao PC.
   b. pseudo-direto.
   c. indireto por registo com deslocamento.
   d. indireto por registo.

2. Considere a instrução `jalr $t9`, armazenada no endereço de memória `0x00400 63C`. Admita que o conteúdo do registo `$t9 = 0x00400680`. Na conclusão da execução da instrução, o registo `$ra`:
   a. terá armazenado o valor `0x00400684`.
   b. terá armazenado o valor `0x0040063C`.
   c. terá armazenado o valor `0x00400640`.
   d. terá armazenado o valor `0x00400680`.

3. Na arquitetura básica de um sistema computacional o *Data Bus*, permite, por exemplo:
   a. especificar a natureza de uma operação efetuada sobre a memória.
   b. identificar, na memória, os dados a serem transferidos.
   c. transferir dados entre os registos do *CPU*.
   d. transferir o código máquina das instruções para o *CPU*.

4. Considere o seguinte segmento de código *assembly*:

```
li   $4,0x011284C3
sw   $4,0x08($2)
lb   $5,0x0A($2)
```

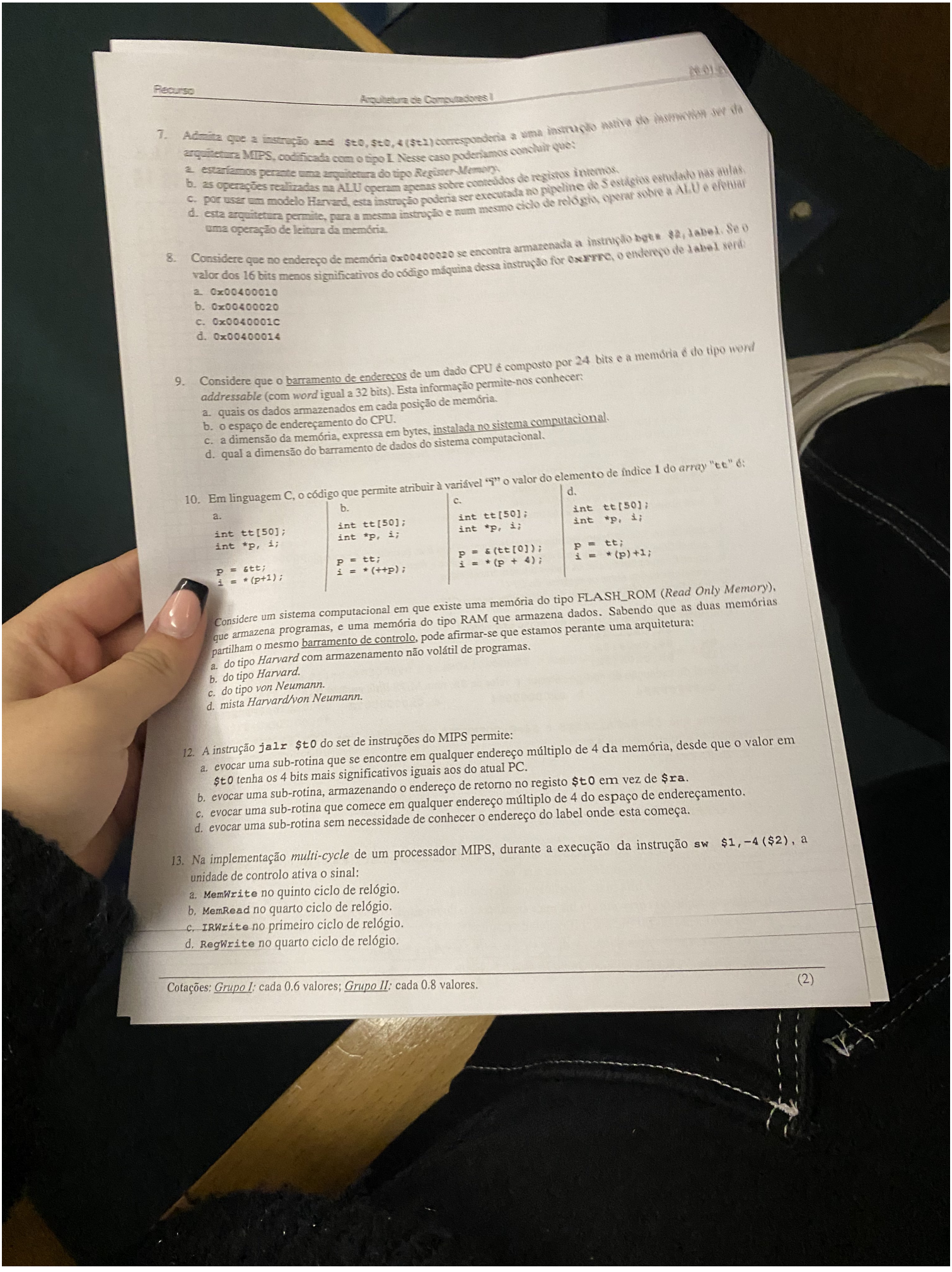
No final da execução do código anterior, o valor de `$5`, num MIPS ***little endian***, é:
a. `0xFFFFFF84` b. `0x00000084` c. `0x00000012` d. `0xFFFFFF01`

5. *Suponha que* `$3=0xFA11AAE8` e `$4=0x00EEFF00`. A instrução "`nor $2,$3,$4`" produz o seguinte resultado armazenado em `$2`:
   a. `0xFAFFAA17`
   b. `0x05000017`
   c. `0x0500FFE8`
   d. `0xFAFFFFE8`

6. A deteção de *overflow* numa operação de adição de dois números inteiros [R=A+B], com R, A e B codificados em complemento para dois, pode fazer-se através:
   a. do ***xor*** entre os **2 bits** mais significativos do resultado.
   b. da avaliação do ***carry out*** do **bit** mais significativo do resultado.
   c. da avaliação do **bit** mais significativo do resultado.
   d. da expressão booleana $R_{n-1} * \overline{A_{n-1}} * \overline{B_{n-1}} + \overline{R_{n-1}} * A_{n-1} * B_{n-1}$.

Cotações: Grupo I: cada 0.6 valores; Grupo II: cada 0.8 valores.

(1)



Recurso — Arquitetura de Computadores I — 26/01/2

7. Admita que a instrução `and $t0,$t0,4($t1)` corresponderia a uma instrução nativa do instruction set da arquitetura MIPS, codificada com o tipo I. Nesse caso poderíamos concluir que:
   a. estaríamos perante uma arquitetura do tipo *Register-Memory*.
   b. as operações realizadas na ALU operam apenas sobre conteúdos de registos internos.
   c. por usar um modelo Harvard, esta instrução poderia ser executada no pipeline de 5 estágios estudado nas aulas.
   d. esta arquitetura permite, para a mesma instrução e num mesmo ciclo de relógio, operar sobre a ALU e efetuar uma operação de leitura da memória.

8. Considere que no endereço de memória `0x00400020` se encontra armazenada a instrução `bgtz $2,label`. Se o valor dos 16 bits menos significativos do código máquina dessa instrução for `0xFFFC`, o endereço de `label` será:
   a. `0x00400010`
   b. `0x00400020`
   c. `0x0040001C`
   d. `0x00400014`

9. Considere que o barramento de endereços de um dado CPU é composto por 24 bits e a memória é do tipo *word addressable* (com *word* igual a 32 bits). Esta informação permite-nos conhecer:
   a. quais os dados armazenados em cada posição de memória.
   b. o espaço de endereçamento do CPU.
   c. a dimensão da memória, expressa em bytes, instalada no sistema computacional.
   d. qual a dimensão do barramento de dados do sistema computacional.

10. Em linguagem C, o código que permite atribuir à variável "i" o valor do elemento de índice 1 do *array* "`tt`" é:

a.
```
int tt[50];
int *p, i;

p = &tt;
i = *(p+1);
```

b.
```
int tt[50];
int *p, i;

p = tt;
i = *(++p);
```

c.
```
int tt[50];
int *p, i;

p = &(tt[0]);
i = *(p + 4);
```

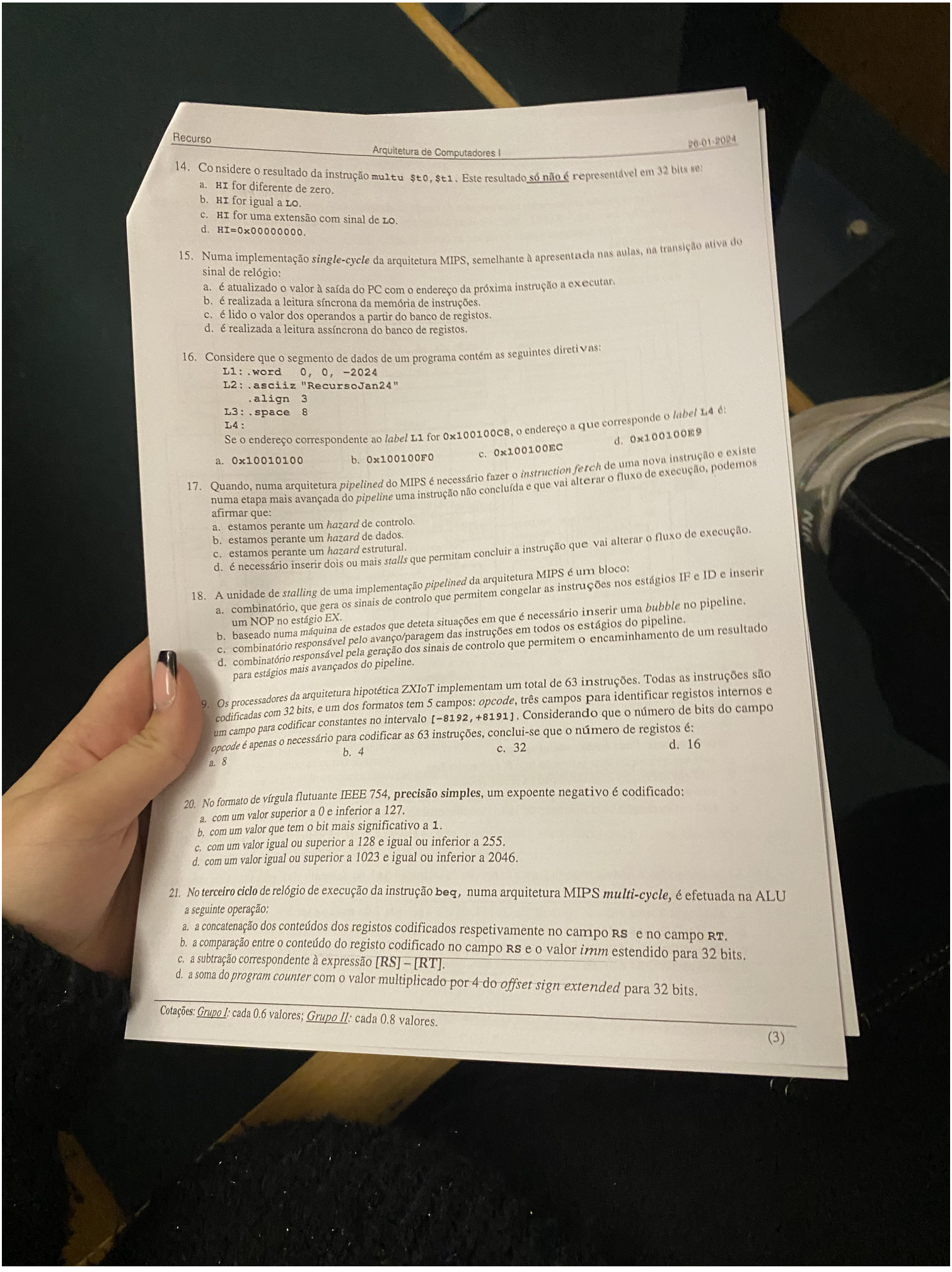
d.
```
int tt[50];
int *p, i;

p = tt;
i = *(p)+1;
```

Considere um sistema computacional em que existe uma memória do tipo FLASH_ROM (*Read Only Memory*), que armazena programas, e uma memória do tipo RAM que armazena dados. Sabendo que as duas memórias partilham o mesmo barramento de controlo, pode afirmar-se que estamos perante uma arquitetura:
a. do tipo *Harvard* com armazenamento não volátil de programas.
b. do tipo *Harvard*.
c. do tipo *von Neumann*.
d. mista *Harvard/von Neumann*.

12. A instrução `jalr $t0` do set de instruções do MIPS permite:
   a. evocar uma sub-rotina que se encontre em qualquer endereço múltiplo de 4 da memória, desde que o valor em `$t0` tenha os 4 bits mais significativos iguais aos do atual PC.
   b. evocar uma sub-rotina, armazenando o endereço de retorno no registo `$t0` em vez de `$ra`.
   c. evocar uma sub-rotina que comece em qualquer endereço múltiplo de 4 do espaço de endereçamento.
   d. evocar uma sub-rotina sem necessidade de conhecer o endereço do label onde esta começa.

13. Na implementação *multi-cycle* de um processador MIPS, durante a execução da instrução `sw $1,-4($2)`, a unidade de controlo ativa o sinal:
   a. `MemWrite` no quinto ciclo de relógio.
   b. `MemRead` no quarto ciclo de relógio.
   c. `IRWrite` no primeiro ciclo de relógio.
   d. `RegWrite` no quarto ciclo de relógio.

Cotações: Grupo I: cada 0.6 valores; Grupo II: cada 0.8 valores. (2)



14. Considere o resultado da instrução `multu $t0,$t1`. Este resultado só não é representável em 32 bits se:
    a. `HI` for diferente de zero.
    b. `HI` for igual a `LO`.
    c. `HI` for uma extensão com sinal de `LO`.
    d. `HI=0x00000000`.

15. Numa implementação ***single-cycle*** da arquitetura MIPS, semelhante à apresentada nas aulas, na transição ativa do sinal de relógio:
    a. é atualizado o valor à saída do PC com o endereço da próxima instrução a executar.
    b. é realizada a leitura síncrona da memória de instruções.
    c. é lido o valor dos operandos a partir do banco de registos.
    d. é realizada a leitura assíncrona do banco de registos.

16. Considere que o segmento de dados de um programa contém as seguintes diretivas:

```
L1: .word   0, 0, -2024
L2: .asciiz "RecursoJan24"
    .align  3
L3: .space  8
L4:
```

Se o endereço correspondente ao *label* `L1` for `0x100100C8`, o endereço a que corresponde o *label* `L4` é:

a. `0x10010100` b. `0x100100F0` c. `0x100100EC` d. `0x100100E9`

17. Quando, numa arquitetura *pipelined* do MIPS é necessário fazer o *instruction fetch* de uma nova instrução e existe numa etapa mais avançada do *pipeline* uma instrução não concluída e que vai alterar o fluxo de execução, podemos afirmar que:
    a. estamos perante um *hazard* de controlo.
    b. estamos perante um *hazard* de dados.
    c. estamos perante um *hazard* estrutural.
    d. é necessário inserir dois ou mais *stalls* que permitam concluir a instrução que vai alterar o fluxo de execução.

18. A unidade de *stalling* de uma implementação *pipelined* da arquitetura MIPS é um bloco:
    a. combinatório, que gera os sinais de controlo que permitem congelar as instruções nos estágios IF e ID e inserir um NOP no estágio EX.
    b. baseado numa *máquina de estados* que deteta situações em que é necessário inserir uma *bubble* no pipeline.
    c. combinatório *responsável pelo avanço/paragem* das instruções em todos os estágios do pipeline.
    d. combinatório *responsável pela geração* dos sinais de controlo que permitem o encaminhamento de um resultado para estágios mais avançados do pipeline.

19. Os processadores da arquitetura hipotética ZXIoT implementam um total de 63 instruções. Todas as instruções são codificadas com 32 bits, e um dos formatos tem 5 campos: *opcode*, três campos para identificar registos internos e um campo para codificar constantes no intervalo `[-8192,+8191]`. Considerando que o número de bits do campo *opcode* é apenas o necessário para codificar as 63 instruções, conclui-se que o número de registos é:

a. 8 b. 4 c. 32 d. 16

20. No formato de vírgula flutuante IEEE 754, **precisão simples**, um expoente negativo é codificado:
    a. com um valor superior a 0 e inferior a 127.
    b. com um valor que tem o bit mais significativo a `1`.
    c. com um valor igual ou superior a 128 e igual ou inferior a 255.
    d. com um valor igual ou superior a 1023 e igual ou inferior a 2046.

21. No **terceiro ciclo** de relógio de execução da instrução `beq`, numa arquitetura MIPS ***multi-cycle***, é efetuada na ALU a seguinte operação:
    a. a concatenação dos conteúdos dos registos codificados respetivamente no campo `RS` e no campo `RT`.
    b. a comparação entre o conteúdo do registo codificado no campo `RS` e o valor *imm* estendido para 32 bits.
    c. a subtração correspondente à expressão [RS] – [RT].
    d. a soma do *program counter* com o valor multiplicado por 4 do *offset sign extended* para 32 bits.

Cotações: Grupo I: cada 0.6 valores; Grupo II: cada 0.8 valores.

Grupo II

22. Admita que os valores armazenados nos elementos de estado do *datapath* da Figura 1, na terceira fase de execução da instrução `lw $2,-8($5)` são: `PC=0x0040005C`, `$5=0x1001OOC8`, `$2=0x00003C24`. Neste pressuposto, o valor disponível à saída do registo `ALUOut`, durante a quarta fase de execução, dessa instrução será:
    a. `0x0040003C`
    b. `0x100100C0`
    c. `0x00400024`
    d. `0x10013C24`

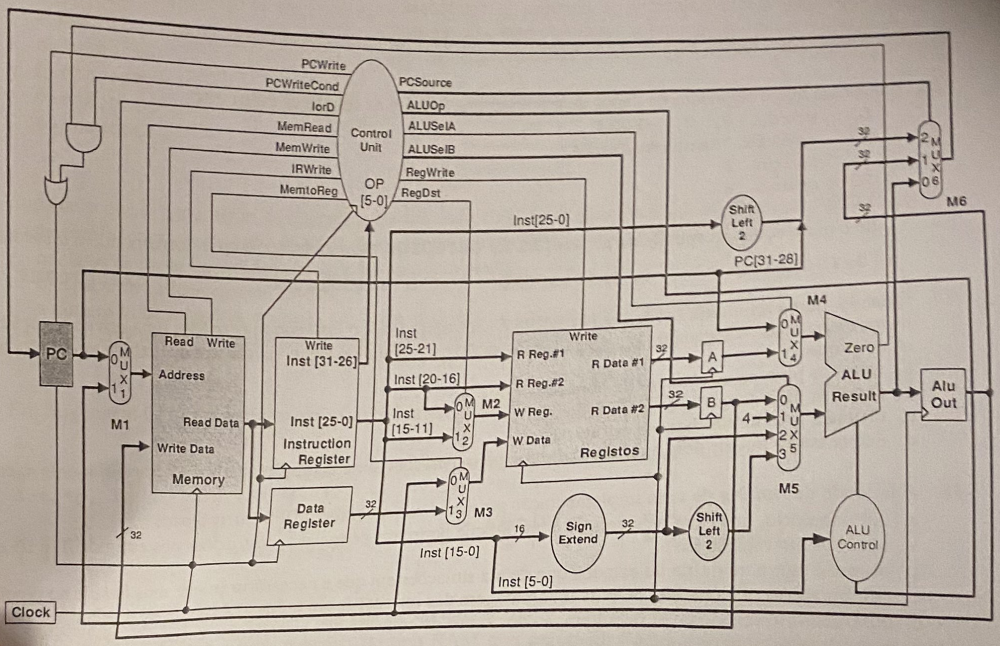


Figura 1

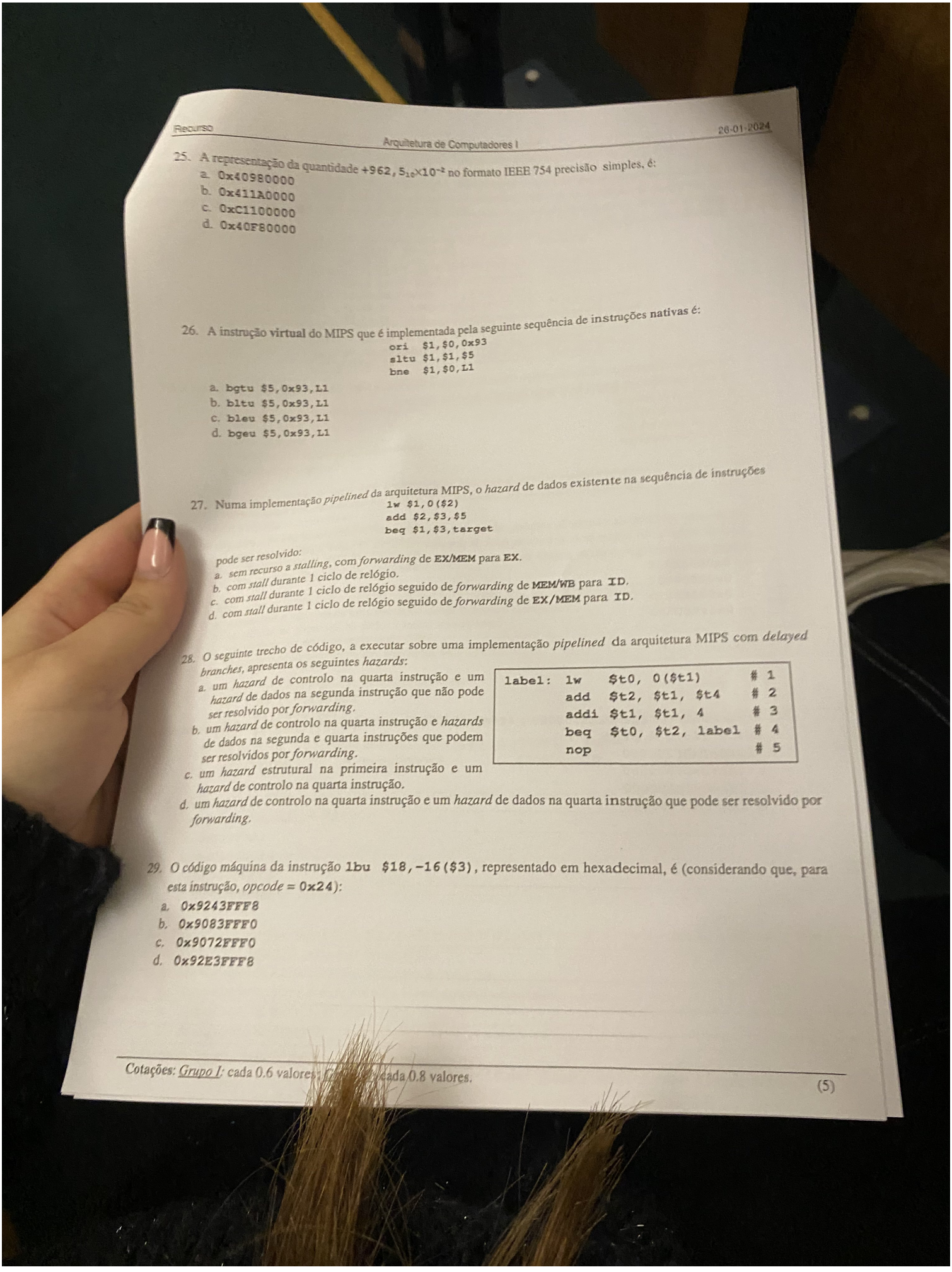
Suponha que, no datapath da Figura 1, imediatamente antes da ocorrência da transição ativa do relógio, os sinais de controlo `PCWrite`, `ALUOp`, `RegDst`, `PCSource` e `RegWrite` têm os valores `'1'`, `"10"`, `'0'`, `"10"` e `'0'` *respetivamente. Pode então concluir-se que:*

a. está em execução uma instrução tipo R na terceira fase.
b. está em execução uma instrução na primeira fase.
c. está em execução uma instrução de salto condicional na terceira fase.
d. está em execução uma instrução de salto incondicional na terceira fase.

24. *Se se pretendesse incluir suporte para a execução da instrução* `jal` *no datapath da Figura 1, uma das alterações a efetuar seria:*

a. acrescentar uma entrada no multiplexer M4, ligada à entrada 2 do multiplexer M6.
b. acrescentar uma entrada no multiplexer M6, ligada à saída do registo A.
c. acrescentar uma entrada no multiplexer M2, ligada à constante **0x04**.
d. acrescentar uma entrada no multiplexer M3, ligada à saída do registo PC.

Cotações: *Grupo I*: cada 0.6 valores; *Grupo II*: cada 0.8 valores.



25. A representação da quantidade $+962,5_{10} \times 10^{-2}$ no formato IEEE 754 precisão simples, é:
    a. `0x40980000`
    b. `0x411A0000`
    c. `0xC1100000`
    d. `0x40F80000`

26. A instrução **virtual** do MIPS que é implementada pela seguinte sequência de instruções **nativas** é:

```
ori  $1,$0,0x93
sltu $1,$1,$5
bne  $1,$0,L1
```

    a. `bgtu $5,0x93,L1`
    b. `bltu $5,0x93,L1`
    c. `bleu $5,0x93,L1`
    d. `bgeu $5,0x93,L1`

27. Numa implementação *pipelined* da arquitetura MIPS, o *hazard* de dados existente na sequência de instruções

```
lw $1,0($2)
add $2,$3,$5
beq $1,$3,target
```

pode ser resolvido:
    a. sem recurso a *stalling*, com *forwarding* de `EX/MEM` para `EX`.
    b. com *stall* durante 1 ciclo de relógio.
    c. com *stall* durante 1 ciclo de relógio seguido de *forwarding* de `MEM/WB` para `ID`.
    d. com *stall* durante 1 ciclo de relógio seguido de *forwarding* de `EX/MEM` para `ID`.

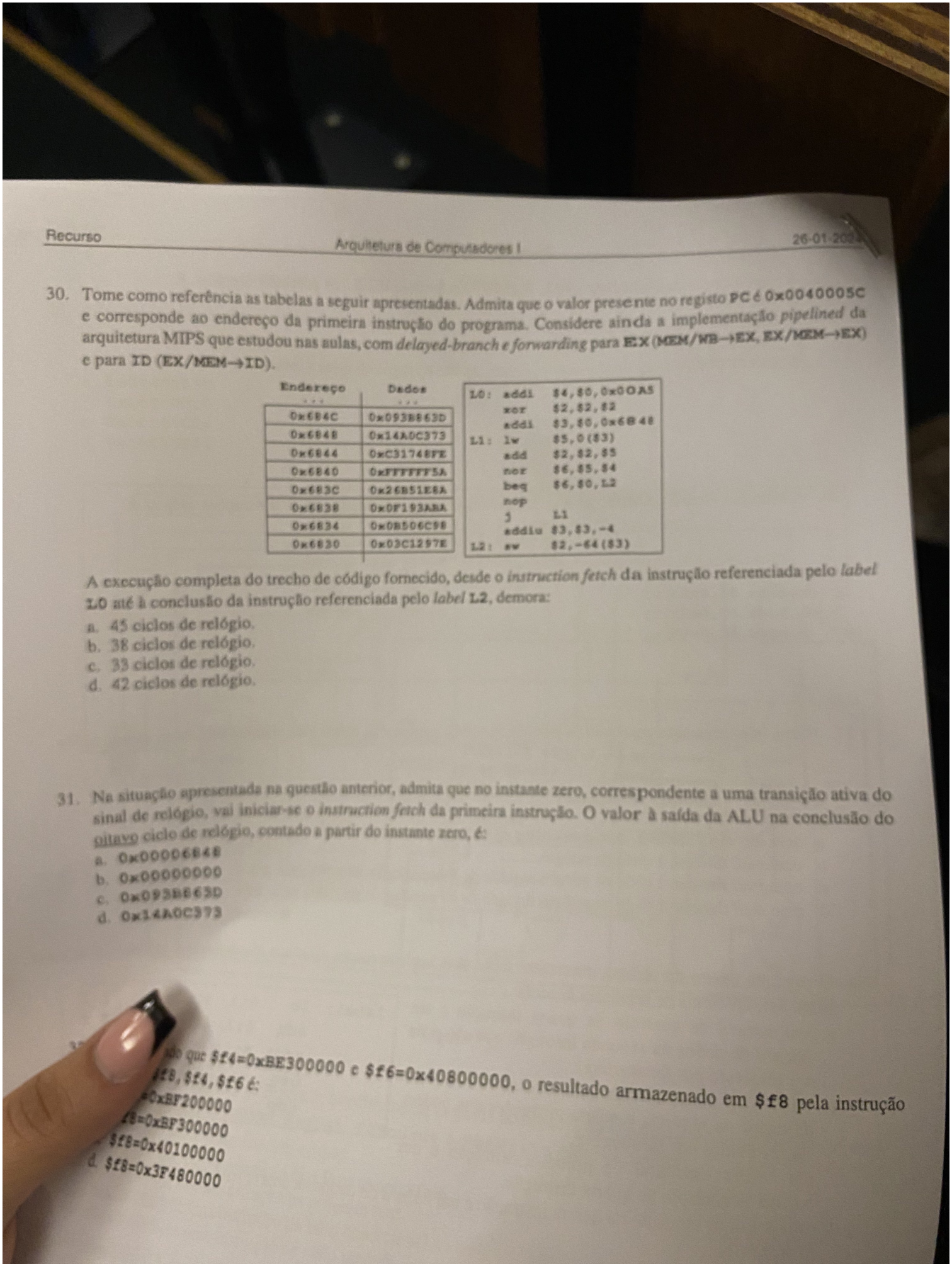
28. O seguinte trecho de código, a executar sobre uma implementação *pipelined* da arquitetura MIPS com *delayed branches*, apresenta os seguintes *hazards*:

```
label:  lw   $t0, 0($t1)      # 1
        add  $t2, $t1, $t4    # 2
        addi $t1, $t1, 4      # 3
        beq  $t0, $t2, label  # 4
        nop                   # 5
```

    a. um *hazard* de controlo na quarta instrução e um *hazard* de dados na segunda instrução que não pode ser resolvido por *forwarding*.
    b. um *hazard* de controlo na quarta instrução e *hazards* de dados na segunda e quarta instruções que podem ser resolvidos por *forwarding*.
    c. um *hazard* estrutural na primeira instrução e um *hazard* de controlo na quarta instrução.
    d. um *hazard* de controlo na quarta instrução e um *hazard* de dados na quarta instrução que pode ser resolvido por *forwarding*.

29. O código máquina da instrução `lbu $18,-16($3)`, representado em hexadecimal, é (considerando que, para esta instrução, *opcode* = `0x24`):
    a. `0x9243FFF8`
    b. `0x9083FFF0`
    c. `0x9072FFF0`
    d. `0x92E3FFF8`

Cotações: Grupo I: cada 0.6 valores; [illegible] cada 0.8 valores.



30. Tome como referência as tabelas a seguir apresentadas. Admita que o valor presente no registo `PC` é `0x0040005C` e corresponde ao endereço da primeira instrução do programa. Considere ainda a implementação *pipelined* da arquitetura MIPS que estudou nas aulas, com *delayed-branch* e *forwarding* para `EX` (`MEM/WB→EX`, `EX/MEM→EX`) e para `ID` (`EX/MEM→ID`).

Endereço	Dados
0x684C	0x093B863D
0x6848	0x14A0C373
0x6844	0xC31748FE
0x6840	0xFFFFFF5A
0x683C	0x26B51E8A
0x6838	0x0F193ABA
0x6834	0x0B506C98
0x6830	0x03C1297E

```
L0:  addi  $4,$0,0x00A5
     xor   $2,$2,$2
     addi  $3,$0,0x6B48
L1:  lw    $5,0($3)
     add   $2,$2,$5
     nor   $6,$5,$4
     beq   $6,$0,L2
     nop
     j     L1
     addiu $3,$3,-4
L2:  sw    $2,-64($3)
```

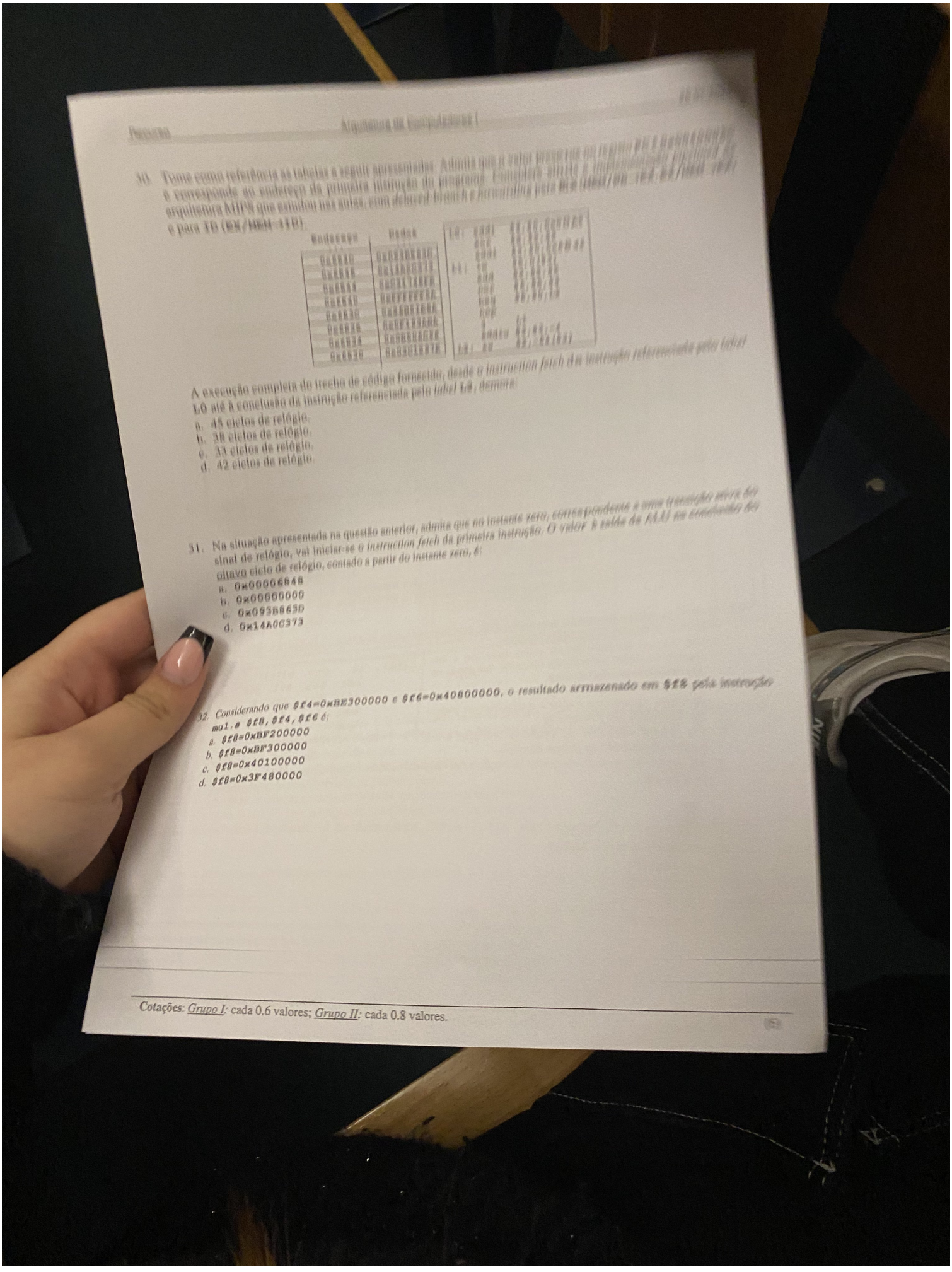
A execução completa do trecho de código fornecido, desde o *instruction fetch* da instrução referenciada pelo *label* `L0` até à conclusão da instrução referenciada pelo *label* `L2`, demora:

a. 45 ciclos de relógio.
b. 38 ciclos de relógio.
c. 33 ciclos de relógio.
d. 42 ciclos de relógio.

31. Na situação apresentada na questão anterior, admita que no instante zero, correspondente a uma transição ativa do sinal de relógio, vai iniciar-se o *instruction fetch* da primeira instrução. O valor à saída da ALU na conclusão do oitavo ciclo de relógio, contado a partir do instante zero, é:

a. `0x00006848`
b. `0x00000000`
c. `0x093B863D`
d. `0x14A0C373`

...do que `$f4=0xBE300000` e `$f6=0x40800000`, o resultado armazenado em `$f8` pela instrução
...$f8,$f4,$f6 é:
...=0xBF200000
...8=0xBF300000
b. `$f8=0x40100000`
d. `$f8=0x3F480000`



Percurso — Arquitetura de Computadores I

30. Tome como referência as tabelas a seguir apresentadas. Admita que o valor presente no registo $t1 é [illegible] e corresponde ao endereço da primeira instrução do programa. Considere ainda a implementação [illegible] arquitetura MIPS que estudou nas aulas, com delayed branch e forwarding para [illegible] e para ID (RX/MEM→ID).

Endereço	Dados
0x8E40	0x8D3B8E3C
0x8E44	0x1A8C0C73
0x8E48	0x0C317A0E8
0x8E4C	0xFFFFFFFA
0x8E50	0xA6B81E0A
0x8E54	0x8B719AA4
0x8E58	0x8B5B4C98
0x8E5C	0x8D3C1E3E

```
L0:  lw   $t0, 0($t1)
     add  $t2, $t0, $t0
     sub  $t3, $t2, $t0
     sw   $t3, 0($t1)
     addi $t1, $t1, -4
     bne  $t1, $t4, L0
L1:  sw   $t0, 0($t1)
```

A execução completa do trecho de código fornecido, desde o instruction fetch da instrução referenciada pela label L0 até à conclusão da instrução referenciada pela label L1, demora:

a. 45 ciclos de relógio.
b. 38 ciclos de relógio.
c. 33 ciclos de relógio.
d. 42 ciclos de relógio.

31. Na situação apresentada na questão anterior, admita que no instante zero, correspondente a uma transição ativa do sinal de relógio, vai iniciar-se o *instruction fetch* da primeira instrução. O valor à saída da ALU no ciclo do oitavo ciclo de relógio, contado a partir do instante zero, é:

a. `0x00006848`
b. `0x00000000`
c. `0x093B863D`
d. `0x14A0C373`

32. Considerando que `$f4=0xBE300000` e `$f6=0x40800000`, o resultado armazenado em `$f8` pela instrução `mul.s $f8,$f4,$f6` é:

a. `$f8=0xBF200000`
b. `$f8=0xBF300000`
c. `$f8=0x40100000`
d. `$f8=0x3F480000`

Cotações: Grupo I: cada 0.6 valores; Grupo II: cada 0.8 valores.

Recurso · 26/01/2024

Chave de respostas publicada com o recurso